

UNIVERSIDAD DEL BOSQUE
Facultad de Ingeniería
Ingeniería de Software & Ingeniería en Robótica

Competitive Programming Notebook

Algoritmos, estructuras de datos y plantillas

C++ · Java · Python

Juan

Estudiante — 4.º semestre
Doble titulación: Software & Robótica

Bogotá, Colombia · 2026

Parte I Fundamentos

1 Entrada / Salida Rápida (Fast I/O)

De más lento (didáctico) a más rápido (para TLE ajustado). Usa el lector por bytes solo cuando haya del orden de 10^6 – 10^7 lecturas.

1.1 C++ (g++ 8.1, -std=c++17)

Tier 1 — Básico: cin / cout.

```
1 int entero; double decimal; string palabra, linea;
2 cin >> entero >> decimal >> palabra; // >> lee UN token
3 // ! cin >> deja el '\n' pendiente;
4 // ! el 1er getline sale VACÍO -> hay que ignorarlo
5 cin.ignore();
6 getline(cin, linea); // línea completa
```

Tier 2 — Rápido: desactiva la sync con C (una vez en main). Punto dulce para casi todo.

```
1 ios_base::sync_with_stdio(false);
2 cin.tie(nullptr); // ! no mezclar con scanf/printf
3 cout << entero << "\n"; // "\n", nunca endl (endl frena)
```

Tier 3 — Más rápido: lector por bytes con fread. Solo con 10^6 – 10^7 lecturas.

```
1 struct FastReader {
2     static const int TAM = 1 << 16; // 64 KB
3     char bufer[TAM];
4     int puntero = 0, leídos = 0;
5
6     int leer() {
7         if (puntero == leídos) {
8             leídos = (int) fread(bufer, 1, TAM, stdin);
9             puntero = 0;
10        }
11        return leídos == 0 ? -1 : bufer[puntero++]; // -1=EOF
12    }
13    int nextInt() {
14        int resultado = 0, b = leer();
15        while (b <= ' ') b = leer(); // saltar blancos
16        bool negativo = (b == '-');
17        if (negativo) b = leer();
18        while (b >= '0' && b <= '9') {
19            resultado = resultado * 10 + (b - '0'); b = leer();
20        }
21        return negativo ? -resultado : resultado;
22    }
23    long long nextLong() {
24        long long resultado = 0; int b = leer();
25        while (b <= ' ') b = leer();
26        bool negativo = (b == '-');
27        if (negativo) b = leer();
28        while (b >= '0' && b <= '9') {
29            resultado = resultado * 10 + (b - '0'); b = leer();
30        }
31        return negativo ? -resultado : resultado;
32    }
33    double nextDouble() {
34        double resultado = 0, divisor = 1; int b = leer();
35        while (b <= ' ') b = leer();
36        bool negativo = (b == '-');
37        if (negativo) b = leer();
38        while (b >= '0' && b <= '9') {
39            resultado = resultado * 10 + (b - '0'); b = leer();
40        }
41        if (b == '.') {
42            b = leer();
43            while (b >= '0' && b <= '9') {
44                resultado += (b - '0') / (divisor *= 10); b = leer();
45            }
46        }
47        return negativo ? -resultado : resultado;
48    }
49    string next() { // un token
50        string s; int b = leer();
51        while (b <= ' ') b = leer();
52        while (b > ' ') { s += (char) b; b = leer(); }
```

```
53        return s;
54    }
55    string nextLine() { // línea completa
56        string s; int b = leer();
57        while (b != '\n' && b != -1) {
58            if (b != '\r') s += (char) b; b = leer();
59        }
60        return s;
61    }
62 }
```

1.2 Java

Tier 1 — Básico: Scanner + System.out.

```
1 Scanner sc = new Scanner(System.in);
2 int entero = sc.nextInt();
3 String palabra = sc.next(); // un token
4 // ! tras nextInt/next, el 1er nextLine sale VACÍO:
5 sc.nextLine(); // quema la línea pendiente
6 String linea = sc.nextLine(); // ahora sí la línea real
```

Tier 2 — Intermedio: BufferedReader + StringTokenizer + PrintWriter. Estándar en CP.

```
1 BufferedReader lector = new BufferedReader(
2     new InputStreamReader(System.in));
3 PrintWriter escritor = new PrintWriter(
4     new BufferedWriter(new OutputStreamWriter(System.out)));
5
6 StringTokenizer separador =
7     new StringTokenizer(lector.readLine());
8 int primero = Integer.parseInt(separador.nextToken());
9 int segundo = Integer.parseInt(separador.nextToken());
10
11 escritor.println(primero + segundo);
12 escritor.flush(); // ! OBLIGATORIO al final, si no, no imprime
```

Tier 3 — StreamTokenizer: veloz para leer números.

```
1 StreamTokenizer in = new StreamTokenizer(
2     new BufferedReader(new InputStreamReader(System.in)));
3 in.nextToken();
4 int entero = (int) in.nval; // ! nval es double: ojo con > 2^53
```

Tier 4 — Más rápido: lector por bytes (DataInputStream). El más veloz sin librerías.

```
1 static class FastReader {
2     private final DataInputStream flujo;
3     private final byte[] bufer = new byte[1 << 16];
4     private int puntero = 0, leídos = 0;
5
6     FastReader() { flujo = new DataInputStream(System.in); }
7
8     int nextInt() throws IOException {
9         int resultado = 0, b = leer();
10        while (b <= ' ') b = leer();
11        boolean negativo = (b == '-');
12        if (negativo) b = leer();
13        while (b >= '0' && b <= '9') {
14            resultado = resultado * 10 + (b - '0'); b = leer();
15        }
16        return negativo ? -resultado : resultado;
17    }
18    long nextLong() throws IOException {
19        long resultado = 0; int b = leer();
20        while (b <= ' ') b = leer();
21        boolean negativo = (b == '-');
22        if (negativo) b = leer();
23        while (b >= '0' && b <= '9') {
24            resultado = resultado * 10 + (b - '0'); b = leer();
25        }
26        return negativo ? -resultado : resultado;
27    }
28    private int leer() throws IOException {
29        if (puntero == leídos) {
30            leídos = flujo.read(bufer, 0, bufer.length);
31            puntero = 0;
32        }
33        return leídos == -1 ? -1 : bufer[puntero++];
34    }
35 }
```

1.3 Python (3.7.4)

Tier 1 — Básico: `input()` / `print()`.

```
1 entero = int(input()) # lee UNA línea
2 primero, segundo = map(int, input().split())
```

Tier 2 — Intermedio: `sys.stdin/stdout` (más rápido que `input()`).

```
1 import sys
2 entrada = sys.stdin.readline
3 salida = sys.stdout.write
4 a, b = map(int, entrada().split())
5 salida(f"{a + b}\n")
```

Tier 3 — Más rápido: leer TODO el `stdin` de golpe y tokenizar; la salida, en UNA sola escritura.

```
1 import sys
2 def main():
3     datos = sys.stdin.buffer.read().split()
4     indice = 0
5     def siguiente_int(): # avanza por los tokens
6         nonlocal indice
7         valor = int(datos[indice]); indice += 1
8         return valor
9
10    n = siguiente_int()
11    total = 0
12    for _ in range(n):
13        total += siguiente_int()
14    sys.stdout.write(f"{total}\n")
15 main()
16
17 # Nota: en Python 32-bit ojo con la memoria (~2 GB).
18 # Para DFS recursivo: sys.setrecursionlimit(300000)
```

2 Formateo de Salida

2.1 C++ (printf)

```
1 printf("%d\n", 42); // 42 entero
2 printf("%lld\n", largo); // long long: SIEMPRE %lld
3 printf("%5d\n", 42); // " 42" ancho 5, derecha
4 printf("%-5d\n", 42); // "42 " izquierda
5 printf("%05d\n", 42); // 00042 rellena ceros
6 printf("%+d\n", 42); // +42 fuerza signo
7 printf("%x %X\n", 255, 255); // ff FF hexadecimal
8 printf("%.2f\n", 3.14159); // 3.14 (lo mas usado en CP)
9 printf("%.0f\n", 3.7); // 4 REDONDEA
10 printf("%e\n", 31415.9); // 3.141590e+04 cientifica
11 printf("%c %s\n", 'A', "hola");
12 printf("100%%\n"); // 100% %% = literal
13 // ! NO uses %n en C (peligroso). scanf: double con %lf
```

2.2 Java (System.out.printf)

```
1 System.out.printf("%d\n", 42); // 42
2 System.out.printf("%,d\n", 1000000); // 1,000,000 miles
3 System.out.printf("%05d\n", 42); // 00042
4 System.out.printf("%+d\n", 42); // +42
5 System.out.printf("%x\n", 255); // ff
6 System.out.printf("%.2f\n", 3.14159); // 3.14
7 System.out.printf("%-10s\n", "hi"); // "hi |"
8 System.out.printf("%b\n", true); // true
9 // %n = salto portable (mejor que \n); %% = %
```

2.3 Python (3 formas)

```
1 # 1) operador % (estilo C)
2 print("%d %.2f" % (42, 3.14))
3 # 2) str.format
4 print("{:05d}".format(42)) # 00042
5 print("{:,}".format(1000000)) # 1,000,000
6 print("{:.1%}".format(0.25)) # 25.0%
7 # 3) f-strings (lo mas limpio, 3.6+)
8 x = 3.14159
9 print(f"{x:.2f}") # 3.14
10 print(f"{42:+d} {255:x}") # +42 ff
```

3 Condicionales y Ciclos

3.1 C++

```
1 if (x > 0) cout << "pos";
2 else if (x < 0) cout << "neg";
3 else cout << "cero";
4
5 string s = (x >= 0) ? "no neg" : "neg"; // ternario
6
7 switch (x) { // solo int/char, NO string
8     case 1: /*...*/ break; // ! sin break: fall-through
9     case 2: case 3: /*...*/ break; // varios case = OR
10    default: /*...*/ ;
11 }
12 if (int r = x % 2; r == 0) {} // C++17: if con init
13
14 for (int i = 0; i < n; i++) {}
15 for (int v : datos) {} // range-based (sin indice)
16 for (int &v : datos) v *= 2; // & modifica el original
17 while (c < n) c++;
18 do { c--; } while (c > 0); // ejecuta AL MENOS 1 vez
19 // break / continue.
20 // ! C++ NO tiene break con etiqueta: usa bandera o return
```

3.2 Java

```
1 if (x > 0) {} else if (x < 0) {} else {}
2
3 String s = (x >= 0) ? "no neg" : "neg"; // ternario
4
5 switch (x) {
6     case 1: /*...*/ break; // ! sin break: fall-through
7     case 2: case 3: /*...*/ break;
8     default: /*...*/ ;
9 }
10
11 for (int i = 0; i < n; i++) {}
12 for (int v : arreglo) {} // for-each
13 while (c < n) c++;
14 do { c--; } while (c > 0); // al menos 1 vez
15
16 externo: // EXCLUSIVO: break con etiqueta
17 for (int i = 0; i < 3; i++)
18     for (int j = 0; j < 3; j++)
19         if (i + j == 3) break externo; // rompe el externo
```

3.3 Python (3.7.4)

```
1 if x > 0:
2     pass
3 elif x < 0: # 'elif', no 'else if'
4     pass
5 else:
6     pass
7
8 s = "no neg" if x >= 0 else "neg" # ternario
9
10 # ! 3.7 NO tiene match/case (3.10). Usa dict de despacho:
11 opciones = {1: "uno", 2: "dos", 3: "tres"}
12 print(opciones.get(x, "otro")) # .get con default
13
14 for i in range(n): # range(inicio, fin, paso), fin excl.
15     pass
16 for v in datos: # for-each idiomático
17     pass
18 for i, v in enumerate(datos): # indice + valor
19     pass
20 while c < n:
21     c += 1
22 while True: # ! Python NO tiene do-while
23     c -= 1
24     if c <= 0:
25         break
26
27 for i in range(5): # EXCLUSIVO: for-else
28     if i == 99:
29         break
30 else: # corre solo si NO hubo break
31     print("sin break")
```

4 Conversiones, Parseos y Casteos

4.1 C++

```

1 // string -> numero (parseo)
2 int e = stoi("42");
3 long long l = stoll("10000000000");
4 double d = stod("3.14");
5 int bin = stoi("1010", nullptr, 2); // 10
6 int hex = stoi("ff", nullptr, 16); // 255
7 // numero -> string
8 string s = to_string(42); // "42"
9 // casteo: (int) TRUNCA hacia 0
10 int t = (int) 3.99; // 3
11 int r = static_cast<int>(3.99); // 3
12 // ! int/int = division ENTERA
13 double mal = 7 / 2; // 3.0
14 double bien = 7 / 2.0; // 3.5
15 // caracteres
16 int cod = (int) 'A'; // 65
17 char ch = (char) 66; // 'B'
18 int dig = '7' - '0'; // 7

```

4.2 Java

```

1 // String -> numero (parseo)
2 int e = Integer.parseInt("42");
3 long l = Long.parseLong("10000000000");
4 double d = Double.parseDouble("3.14");
5 int bin = Integer.parseInt("1010", 2); // 10
6 int hex = Integer.parseInt("ff", 16); // 255
7 // numero -> String
8 String s = String.valueOf(42); // "42"
9 String h = Integer.toHexString(255); // "ff"
10 // casteo
11 int t = (int) 3.99; // 3 (TRUNCA)
12 int r = (int) Math.round(3.99); // 4
13 double mal = 7 / 2; // 3.0
14 double bien = 7 / 2.0; // 3.5
15 // caracteres
16 int cod = (int) 'A'; // 65
17 char ch = (char) 66; // 'B'
18 int dig = '7' - '0'; // 7

```

4.3 Python (3.7.4)

```

1 # string -> numero
2 e = int("42"); d = float("3.14")
3 bin = int("1010", 2) # 10
4 hex = int("ff", 16) # 255
5 e2 = int(float("3.99")) # 3 (int("3.14") da ERROR)
6 # numero -> string
7 s = str(42) # "42"
8 b = format(10, "b") # "1010" (sin prefijo)
9 h = format(255, "x") # "ff"
10 # casteo
11 t = int(3.99) # 3 (TRUNCA hacia 0)
12 r = round(3.99) # 4
13 piso = 7 // 2 # 3
14 real = 7 / 2 # 3.5 (siempre float)
15 # caracteres
16 cod = ord("A") # 65
17 ch = chr(66) # "B"
18 dig = ord("7") - ord("0") # 7

```

Parte II Estructuras de Datos

5 Estructuras de Datos (Data Structures)

5.1 Arreglos (Arrays)

Colección de tamaño **fijo** del mismo tipo. **Complejidad:** acceso/asignación por índice $O(1)$; crear $O(n)$.

```

1 int a[5] = {1,2,3,4,5}; // crear: O(n)
2 a[0] = 10; // acceso/asignación: O(1)

```

```

1 int[] a = new int[5]; // crear: O(n)
2 a[0] = 10; // acceso: O(1)
3 int n = a.length; // O(1) (.length sin ())

```

```

1 a = [0] * 5 # crear: O(n) (no hay arreglo fijo)
2 a[0] = 10 # acceso: O(1)

```

5.2 Matrices (Arreglos 2D)

Arreglo bidimensional (filas $\times$ columnas). **Complejidad:** acceso $[i][j]$ $O(1)$; crear $n \times m$ en $O(nm)$.

```

1 vector<vector<int>> m(3, vector<int>(4,0)); // crear: O(n*m)
2 m[1][2] = 7; // acceso: O(1)

```

```

1 int[] [] m = new int[3][4]; // crear: O(n*m)
2 m[1][2] = 7; // acceso: O(1)

```

```

1 m = [[0]*4 for _ in range(3)] # crear: O(n*m)
2 m[1][2] = 7 # acceso: O(1)
3 # ! NO [[0]*4]*3 (las filas se comparten)

```

5.3 Vectores / Listas dinámicas

Arreglo que **crece** solo (ArrayList / vector / list). **Complejidad:** acceso por índice $O(1)$; agregar al final $O(1)$ amortizado; insertar/borrar en medio $O(n)$; buscar $O(n)$.

```

1 vector<int> v;
2 v.push_back(3); // final: O(1) amortizado
3 int x = v[0]; // acceso: O(1)
4 int n = v.size(); // O(1)

```

```

1 ArrayList<Integer> v = new ArrayList<>();
2 v.add(3); // final: O(1) amortizado
3 int x = v.get(0); // acceso: O(1)
4 int n = v.size(); // O(1)

```

```

1 v = []
2 v.append(3) # final: O(1) amortizado
3 x = v[0] # acceso: O(1)
4 n = len(v) # O(1)

```

5.4 Listas enlazadas (LinkedList)

Nodos enlazados. **Complejidad:** insertar/eliminar en los **extremos** $O(1)$; acceso por índice $O(n)$; buscar $O(n)$. Útil como cola o deque.

```

1 list<int> l; // o deque<int>
2 l.push_front(1); // O(1)
3 l.push_back(9); // O(1) acceso [i]: O(n)

```

```

1 LinkedList<Integer> l = new LinkedList<>();
2 l.addFirst(1); // O(1)
3 l.addLast(9); // O(1)
4 l.removeFirst(); // O(1) get(i): O(n)

```

```

1 from collections import deque
2 d = deque()
3 d.appendleft(1) # O(1)
4 d.append(9) # O(1)
5 d.popleft() # O(1) d[i]: O(n)

```

5.5 Conjuntos hash (HashSet)

Colección **sin duplicados** y sin orden. **Complejidad:** insertar, buscar y borrar $O(1)$ promedio ($O(n)$ peor caso).

```
1 unordered_set<int> s;
2 s.insert(3); // O(1) prom.
3 bool hay = s.count(3); // O(1) prom.
```

```
1 HashSet<Integer> s = new HashSet<>();
2 s.add(3); // O(1) prom.
3 boolean hay = s.contains(3); // O(1) prom.
```

```
1 s = set()
2 s.add(3) # O(1) prom.
3 hay = 3 in s # O(1) prom.
```

5.6 Mapas / Diccionarios hash

Pares **clave** → **valor** sin orden (HashMap / unordered_map / dict). **Complejidad:** insertar, acceder y borrar por clave $O(1)$ promedio ($O(n)$ peor caso).

```
1 unordered_map<string,int> m;
2 m["a"] = 1; // insertar/asignar: O(1) prom.
3 int v = m["a"]; // acceso: O(1) prom.
```

```
1 HashMap<String,Integer> m = new HashMap<>();
2 m.put("a", 1); // O(1) prom.
3 int v = m.getOrDefault("a", 0); // O(1) prom.
```

```
1 m = {}
2 m["a"] = 1 # O(1) prom.
3 v = m.get("a", 0) # O(1) prom. (get con default)
```

5.7 Conjuntos ordenados (TreeSet)

Conjunto sin duplicados **mantenido ordenado** (árbol balanceado). **Complejidad:** insertar, buscar, borrar y piso/techo $O(\log n)$.

```
1 set<int> s;
2 s.insert(5); s.insert(1); // O(log n) c/u
3 int menor = *s.begin(); // min: O(1)
4 auto it = s.lower_bound(3); // techo: O(log n)
```

```
1 TreeSet<Integer> s = new TreeSet<>();
2 s.add(5); s.add(1); // O(log n) c/u
3 int menor = s.first(); // O(log n)
4 Integer techo = s.ceiling(3); // O(log n)
```

```
1 # ! Python NO trae TreeSet built-in
2 from sortedcontainers import SortedSet
3 s = SortedSet()
4 s.add(5); s.add(1) # O(log n) c/u
5 menor = s[0] # min: O(log n)
6 i = s.bisect_left(3) # techo: O(log n)
```

5.8 Mapas ordenados (TreeMap)

Mapa **clave** → **valor ordenado por clave** (árbol balanceado). **Complejidad:** insertar, acceder, borrar y floor/ceiling $O(\log n)$.

```
1 map<string,int> m;
2 m["b"] = 2; m["a"] = 1; // O(log n) c/u
3 auto primera = m.begin(); // clave menor: O(1)
```

```
1 TreeMap<String,Integer> m = new TreeMap<>();
2 m.put("b", 2); m.put("a", 1); // O(log n) c/u
3 String prim = m.firstKey(); // O(log n)
```

```
1 # ! dict NO ordena por clave (conserva insercion)
2 from sortedcontainers import SortedDict
3 m = SortedDict()
4 m["b"] = 2; m["a"] = 1 # O(log n) c/u
5 prim = m.peekitem(0) # menor clave: O(log n)
```

5.9 Pilas (Stack)

Estructura **LIFO** (último en entrar, primero en salir). **Complejidad:** push, pop y consultar el tope $O(1)$. Para DFS iterativo, paréntesis balanceados, deshacer.

```
1 stack<int> pila;
2 pila.push(3); // O(1)
3 int tope = pila.top(); // O(1)
4 pila.pop(); // O(1)
```

```
1 Deque<Integer> pila = new ArrayDeque<>(); // no uses Stack
2 pila.push(3); // O(1)
3 int tope = pila.peek(); // O(1)
4 pila.pop(); // O(1)
```

```
1 pila = []
2 pila.append(3) # push: O(1)
3 tope = pila[-1] # O(1)
4 pila.pop() # O(1)
```

5.10 Colas (Queue)

Estructura **FIFO** (primero en entrar, primero en salir). **Complejidad:** encolar y desencolar $O(1)$. Base del BFS.

```
1 queue<int> cola;
2 cola.push(3); // encolar: O(1)
3 int frente = cola.front(); // O(1)
4 cola.pop(); // desencolar: O(1)
```

```
1 Queue<Integer> cola = new ArrayDeque<>();
2 cola.offer(3); // encolar: O(1)
3 int frente = cola.peek(); // O(1)
4 cola.poll(); // desencolar: O(1)
```

```
1 from collections import deque
2 cola = deque()
3 cola.append(3) # encolar: O(1)
4 frente = cola[0] # O(1)
5 cola.popleft() # desencolar: O(1)
```

5.11 Colas de prioridad (Priority Queue / Heap)

Devuelve siempre el elemento de mayor (o menor) prioridad. **Complejidad:** insertar y extraer $O(\log n)$; consultar el tope $O(1)$. Para Dijkstra, Prim, k-ésimo mayor.

```
1 priority_queue<int> pq; // MAX-heap por defecto
2 pq.push(3); // O(log n)
3 int mx = pq.top(); // O(1) (el mayor)
4 pq.pop(); // O(log n)
5 // min-heap:
6 priority_queue<int, vector<int>, greater<int>> pqMin;
```

```
1 PriorityQueue<Integer> pq = new PriorityQueue<>(); // MIN-heap
2 pq.offer(3); // O(log n)
3 int mn = pq.peek(); // O(1) (el menor)
4 pq.poll(); // O(log n)
5 // max-heap: new PriorityQueue<>(Collections.reverseOrder())
```

```
1 import heapq
2 pq = []
3 heapq.heappush(pq, 3) # O(log n)
4 mn = pq[0] # O(1) (el menor; heapq es MIN-heap)
5 heapq.heappop(pq) # O(log n)
6 # max-heap: guarda los valores negados (-x)
```

5.12 Vector circular (Circular Buffer)

Idea: un arreglo de tamaño fijo (**cap**) que se comporta como una **cola doble**: guardas dónde empieza (**ini**) y cuántos elementos hay (**tam**), y cada posición lógica **i** se traduce a física con módulo: $(ini+i)\%cap$. Así, agregar

o sacar por *cualquiera* de los dos extremos es $O(1)$ *sin mover datos* (en un arreglo normal, insertar al frente cuesta $O(n)$ porque hay que correr todo). Al llenarse, un `pushBack` pisa al más viejo. Es la base de colas de tamaño fijo, ventanas deslizantes y buffers de audio/red.

Complejidad: todo $O(1)$.

```
1 struct VectorCircular {
2     vector<int> buf; int cap, ini=0, tam=0;
3     VectorCircular(int c):buf(c),cap(c){}
4     void pushBack(int x){ buf[(ini+tam)%cap]=x;
5         if(tam<cap) tam++; else ini=(ini+1)%cap; } // O(1)
6     void pushFront(int x){ ini=(ini-1+cap)%cap;
7         buf[ini]=x; if(tam<cap) tam++; } // O(1)
8     int popFront(){ int v=buf[ini];
9         ini=(ini+1)%cap; tam--; return v; } // O(1)
10    int popBack(){ tam--; return buf[(ini+tam)%cap]; }
11    int front(){ return buf[ini]; }
12    int back(){ return buf[(ini+tam-1)%cap]; }
13 };
```

```
1 void pushBack(int x){ buf[(ini+tam)%cap]=x;
2     if(tam<cap) tam++; else ini=(ini+1)%cap; }
3 void pushFront(int x){ ini=(ini-1+cap)%cap;
4     buf[ini]=x; if(tam<cap) tam++; }
5 int popFront(){ int v=buf[ini];
6     ini=(ini+1)%cap; tam--; return v; }
```

```
1 def push_back(self, x):
2     self.buf[(self.ini+self.tam)%self.cap] = x
3     if self.tam < self.cap: self.tam += 1
4     else: self.ini = (self.ini+1) % self.cap
5 def push_front(self, x):
6     self.ini = (self.ini-1) % self.cap
7     self.buf[self.ini] = x
8     if self.tam < self.cap: self.tam += 1
```

5.13 Árbol de Fenwick (BIT)

Idea: un arreglo normal da suma de prefijo en $O(1)$ pero actualizar un elemento cuesta $O(n)$ (o al revés si guardas prefijos). El Fenwick equilibra ambas a $O(\log n)$ guardando *sumas parciales inteligentes*: la posición i almacena la suma de un bloque de tamaño i & $(-i)$ (su bit menos significativo) que termina en i . Para consultar un prefijo saltas hacia atrás quitando el bit bajo ($i -= i \& (-i)$); para actualizar subes sumándolo ($i += i \& (-i)$). Cada camino toca a lo más $\log n$ posiciones (un bit por paso). Es 1-indexado.

Básico: update puntual y suma de prefijo; suma de rango $[l, r] = \text{query}(r) - \text{query}(l-1)$.

```
1 void update(int i, long long v){ // suma v en pos i. O(log
2     for(; i<=n; i+=i&(-i)) t[i]+=v; }
3 long long query(int i){ long long s=0; // suma [1..i]. O(log
4     for(; i>0; i-=i&(-i)) s+=t[i]; return s; }
5 long long rango(int l, int r){ return query(r)-query(l-1); }
```

```
1 void update(int i, long v){
2     for(; i<=n; i+=i&(-i)) t[i]+=v; } // O(log n)
3 long query(int i){ long s=0;
4     for(; i>0; i-=i&(-i)) s+=t[i]; return s; }
```

```
1 def update(self, i, v): # O(log n)
2     while i <= self.n:
3         self.t[i] += v; i += i & (-i)
4 def query(self, i): # suma [1..i], O(log n)
5     s = 0
6     while i > 0:
7         s += self.t[i]; i -= i & (-i)
8     return s
```

Construir en $O(n)$: en vez de n updates ($O(n \log n)$), propaga cada valor a su “padre” una sola vez.

```
1 for(int i=1; i<=n; i++){ t[i]+=a[i-1];
2     int j=i+(i&(-i)); if(j<=n) t[j]+=t[i]; } // O(n)
```

k -ésimo elemento (lower_bound sobre prefijos): si el BIT cuenta frecuencias, con *binary lifting* bajas por el árbol y encuentras el menor índice cuyo prefijo alcanza objetivo, en $O(\log n)$ (mucho mejor que binaria+query, que sería $O(\log^2 n)$).

```
1 int lowerBound(long long objetivo){ // O(log n)
2     int pos=0; long long ac=0, LOG=1;
3     while((1<<LOG)<=n) LOG++;
4     for(int k=LOG-1; k>=0; k--){ int nx=pos+(1<<k);
5         if(nx<=n && ac+t[nx]<objetivo){ pos=nx; ac+=t[nx]; } }
6     return pos+1; }
```

Update de rango + query de rango: con *dos* BIT y el truco de diferencias, “sumar v a todo $[l, r]$ ” y consultar sumas de rango, ambas $O(\log n)$.

```
1 void rangoUpdate(int l, int r, long long v){ // suma v a [l, r]
2     b1.update(l, v); b1.update(r+1, -v);
3     b2.update(l, v*(1-1)); b2.update(r+1, -v*(1-1)); }
4 long long prefijo(int i){ return b1.query(i)*i - b2.query(i); }
5 long long rango(int l, int r){ return prefijo(r)-prefijo(l-1); }
```

(Java y Python son análogos, línea por línea; están completos en el archivo `EstructurasAvanzadas` del repo.)

5.14 Sparse Table (RMQ)

Idea: responde *Range Minimum Query* (el mínimo de cualquier rango) en $O(1)$, sobre un arreglo que **no cambia**. Precalcula $\text{sp}[k][i]$ = mínimo del bloque que empieza en i y tiene largo 2^k . La clave: para el rango $[l, r]$ tomas $k = \lfloor \log_2(r-l+1) \rfloor$ y lo cubres con *dos* bloques de largo 2^k —uno pegado a l y otro pegado a r — que **se solapan**; como el mínimo es *idempotente* ($\min(x, x) = x$), contar el solape dos veces no cambia el resultado, así que la consulta son solo dos lecturas. Sirve igual para máximo o gcd (cambiando $\min$); para saber *qué índice* es el mínimo, guarda índices en vez de valores.

Complejidad: construir $O(n \log n)$, consulta $O(1)$. (Si el arreglo *cambia*, usa Segment Tree: $O(\log n)/\text{op.}$)

```
1 sp[0]=a; // build: O(n log n)
2 for(int k=1; k<K; k++) for(int i=0; i+(1<<k)<=n; i++){
3     sp[k][i]=min(sp[k-1][i], sp[k-1][i+(1<<(k-1))]);
4 int minRango(int l, int r){ // O(1)
5     int k=lg[r-l+1];
6     return min(sp[k][l], sp[k][r-(1<<k)+1]); }
```

```
1 for k in range(1, K): # build: O(n log n)
2     for i in range(n-(1<<k)+1):
3         sp[k][i]=min(sp[k-1][i], sp[k-1][i+(1<<(k-1))])
4 def min_rango(l, r): # O(1)
5     k = lg[r-l+1]
6     return min(sp[k][l], sp[k][r-(1<<k)+1])
```

(En Java es idéntico al C++ con `Math.min`.)

Parte III Ordenamiento y Búsqueda

6 Ordenamiento y Búsqueda (Sorting & Searching)

Varias búsquedas lineales y sub-lineales. (La búsqueda binaria tiene su propia sección.)

6.1 Número faltante (Missing Number)

Idea: los números $0, 1, \dots, n$ deberían sumar $\frac{n(n+1)}{2}$ (fórmula de Gauss). Si falta uno, la suma real queda

corta *exactamente* en ese valor, así que el faltante es (suma esperada — suma real). Basta un recorrido para acumular la suma. Alternativa sin riesgo de overflow: hacer XOR de $0..n$ y de todos los elementos; lo que quede es el faltante. **Complejidad:** $O(n)$.

```
1 long long esp = (long long)n*(n+1)/2; // suma 0..n
2 long long falta = esp - accumulate(a.begin(), a.end(), 0LL);
```

```
1 long falta = (long)n*(n+1)/2;
2 for (int x : a) falta -= x;
```

```
1 falta = n*(n+1)/2 - sum(a)
```

6.2 Máximo / Mínimo (Array Max/Min)

Idea: recorres el arreglo una sola vez llevando el menor y el mayor vistos hasta ahora; comparas cada elemento contra ambos y actualizas. No requiere ordenar (ordenar sería $O(n \log n)$, peor). Con un recorrido consigues los dos a la vez. **Complejidad:** $O(n)$.

```
1 int mn = *min_element(a.begin(), a.end());
2 int mx = *max_element(a.begin(), a.end());
```

```
1 int mn = a[0], mx = a[0];
2 for (int x : a) { mn = Math.min(mn, x); mx = Math.max(mx, x); }
```

```
1 mn, mx = min(a), max(a)
```

6.3 Par de suma absoluta mínima

Idea: buscamos el par cuya suma esté *más cerca de cero*. Tras ordenar, ponemos un puntero en cada extremo. La suma de los extremos te dice hacia dónde moverte: si es negativa, para acercarla a 0 hay que subir el menor ($lo++$); si es positiva, hay que bajar el mayor ($hi--$). Cada movimiento descarta el candidato menos prometedor, y en el camino guardas el $|suma|$ más pequeño visto. Los dos punteros recorren el arreglo una vez ($O(n)$); domina el sort. **Complejidad:** $O(n \log n)$.

```
1 sort(a.begin(), a.end());
2 int lo=0, hi=a.size()-1; long long mej=LLONG_MAX;
3 while(lo<hi){ long long s=(long long)a[lo]+a[hi];
4     mej=min(mej, llabs(s));
5     if(s<0) lo++; else hi--; }
```

```
1 Arrays.sort(a);
2 int lo=0, hi=a.length-1; long mej=Long.MAX_VALUE;
3 while(lo<hi){ long s=(long)a[lo]+a[hi];
4     mej=Math.min(mej, Math.abs(s));
5     if(s<0) lo++; else hi--; }
```

```
1 a.sort(); lo, hi = 0, len(a)-1; mej = float("inf")
2 while lo < hi:
3     s = a[lo] + a[hi]; mej = min(mej, abs(s))
4     if s < 0: lo += 1
5     else: hi -= 1
```

6.4 Par con diferencia dada (Difference Pair)

Idea: ¿existe un par cuya diferencia sea exactamente d ? Tras ordenar, dos punteros avanzan hacia adelante: si la diferencia actual $a_j - a_i$ es menor que d , adelantas el puntero lejano (j) para agrandarla; si es mayor, adelantas el cercano (i) para achicarla; si es igual, encontraste el par. Como el arreglo está ordenado, la diferencia crece con j y decrece con i , así que el barrido nunca retrocede. **Complejidad:** $O(n \log n)$ (domina el sort).

```
1 sort(a.begin(), a.end()); int i=0, j=1, n=a.size();
2 while(i<n && j<n){
3     if(i!=j && a[j]-a[i]==d) return true;
4     if(a[j]-a[i]<d) j++; else i++; }
```

```
1 Arrays.sort(a); int i=0, j=1, n=a.length;
2 while(i<n && j<n){
3     if(i!=j && a[j]-a[i]==d) return true;
4     if(a[j]-a[i]<d) j++; else i++; }
```

```
1 a.sort(); i, j, n = 0, 1, len(a)
2 while i < n and j < n:
3     if i != j and a[j]-a[i] == d: return True
4     if a[j]-a[i] < d: j += 1
5     else: i += 1
```

6.5 Búsqueda ternaria (Ternary Search)

Idea: sirve para hallar el pico de una función **unimodal** (sube y luego baja, con un solo máximo — no para buscar un valor en un arreglo cualquiera). Partes el rango en tres con dos puntos $m_1 < m_2$ y comparas: si $f(m_1) < f(m_2)$, el máximo no puede estar a la izquierda de m_1 , así que descartas ese tercio; en caso contrario descartas el tercio derecho. Cada paso elimina $\sim \frac{1}{3}$ del rango, por eso converge en logarítmico. (Para un mínimo, invierte la comparación.) **Complejidad:** $O(\log n)$ — NO $O(n \log n)$.

```
1 while(hi-lo>2){ long long m1=lo+(hi-lo)/3, m2=hi-(hi-lo)/3;
2     if(f(m1)<f(m2)) lo=m1+1; else hi=m2-1; }
3 // revisa [lo,hi] y devuelve el x que maximiza f
```

```
1 while(hi-lo>2){ long m1=lo+(hi-lo)/3, m2=hi-(hi-lo)/3;
2     if(f(m1)<f(m2)) lo=m1+1; else hi=m2-1; }
```

```
1 while hi - lo > 2:
2     m1 = lo + (hi-lo)//3; m2 = hi - (hi-lo)//3
3     if f(m1) < f(m2): lo = m1 + 1
4     else: hi = m2 - 1
```

6.6 Búsqueda de Fibonacci (Fibonacci Search)

Idea: misma estrategia que la binaria (descartar mitades de un arreglo ordenado), pero en vez de partir por el punto medio elige el punto de comparación usando números de Fibonacci consecutivos. Mantiene tres Fibonacci y en cada paso mira la posición $off+f_2$; según el elemento sea menor o mayor que el objetivo, retrocede uno o dos pasos en la secuencia. Ventaja: solo usa *sumas y restas*, nunca división, lo que la hace útil en hardware sin división rápida o para saltar por posiciones amigables con la caché. **Complejidad:** $O(\log n)$.

```
1 int f2=0, f1=1, fb=f1+f2;
2 while(fb<n){ f2=f1; f1=fb; fb=f1+f2; }
3 int off=-1;
4 while(fb>1){ int i=min(off+f2, n-1);
5     if(a[i]<x){ fb=f1; f1=f2; f2=fb-f1; off=i; }
6     else if(a[i]>x){ fb=f2; f1=f1-f2; f2=fb-f1; }
7     else return i; }
8 if(f1==1 && off+1<n && a[off+1]==x) return off+1;
```

```
1 f2, f1 = 0, 1; fb = f1 + f2
2 while fb < n: f2, f1 = f1, fb; fb = f1 + f2
3 off = -1
4 while fb > 1:
5     i = min(off + f2, n-1)
6     if a[i] < x: fb, f1, f2, off = f1, f2, f1 - f2, i
7     elif a[i] > x: fb, f1, f2 = f2, f1 - f2, fb - f1
8     else: return i
```

(En Java es idéntico al C++, cambiando `a.length`.)

6.7 Búsqueda exponencial (Exponential Search)

Idea: dos fases sobre un arreglo **ordenado**. Primero *acotas* el objetivo doblando el índice (1, 2, 4, 8, ...) hasta que $a[i]$ lo supera; en ese momento sabes que el objetivo está entre $i/2$ e i . Segundo, haces una binaria normal dentro de ese bloque. Como el bloque encontrado tiene tamaño proporcional a la posición del objetivo, es ideal cuando el arreglo es enorme o de tamaño desconocido y el objetivo está cerca del inicio. **Complejidad:** $O(\log n)$.

```
1 if(a[0]==x) return 0;
2 int i=1; while(i<n && a[i]<=x) i*=2; // dobla el rango
3 // luego binaria en [i/2, min(i,n-1)]
```

```
1 if a[0] == x: return 0
2 i = 1
3 while i < n and a[i] <= x: i *= 2
4 # luego binaria en [i//2, min(i, n-1)]
```

6.8 Búsqueda por saltos (Jump Search)

Idea: en vez de revisar elemento por elemento, avanzas en bloques de tamaño $\sqrt{n}$ mirando solo el *último* de cada bloque; cuando ese último ya supera al objetivo, sabes que (si existe) está en el bloque anterior, así que lo barres linealmente. El balance $\sqrt{n}$ saltos + $\sqrt{n}$ del barrido minimiza el total. Punto medio entre la lineal ($O(n)$) y la binaria ($O(\log n)$), útil cuando saltar hacia atrás es costoso. **Complejidad:** $O(\sqrt{n})$.

```
1 int paso=max(1,(int)sqrt((double)n)), prev=0;
2 while(prev<n && a[min(prev+paso,n)-1]<x) prev+=paso;
3 for(int i=prev; i<min(prev+paso,n); i++)
4 if(a[i]==x) return i;
```

```
1 import math
2 paso = max(1, int(math.sqrt(n))); prev = 0
3 while prev < n and a[min(prev+paso, n)-1] < x: prev += paso
4 for i in range(prev, min(prev+paso, n)):
5 if a[i] == x: return i
```

6.9 Heap Sort

Idea: un *heap* binario se representa en el mismo arreglo: el hijo izquierdo de i es $2i+1$ y el derecho $2i+2$; en un *max-heap* todo padre es $\geq$ sus hijos, así que el mayor está en la raíz. Heap Sort tiene dos fases. (1) **Construir el heap:** recorres de abajo hacia arriba aplicando *bajar* (sift-down) a cada padre. Aunque parezca $O(n \log n)$, es $O(n)$: casi todos los nodos están cerca de las hojas y se hunden poco (la suma $\sum n/2^h \cdot h$ converge a $2n$). (2) **Ordenar:** repetidamente intercambias la raíz (el mayor) con el último elemento activo —quedando ya ordenado al final— y “hundes” la nueva raíz para restaurar el heap; cada una de las n extracciones cuesta $O(\log n)$. In-place, sin memoria extra, pero **no estable**. **Complejidad:** $O(n \log n)$ en todos los casos.

```
1 void bajar(vector<int>&a,int n,int i){ // hunde: O(log n)
2 while(true){ int m=i,l=2*i+1,r=2*i+2;
3 if(l<n&&a[l]>a[m]) m=l;
4 if(r<n&&a[r]>a[m]) m=r;
5 if(m==i) break; swap(a[i],a[m]); i=m; } }
6 void heapSort(vector<int>&a){ int n=a.size();
7 for(int i=n/2-1;i>=0;i--) bajar(a,n,i); // heap: O(n)
8 for(int i=n-1;i>0;i--){ swap(a[0],a[i]); // max al final
9 bajar(a,i,0); } }
```

```
1 def bajar(a, n, i): # hunde: O(log n)
2 while True:
3     m, l, r = i, 2*i+1, 2*i+2
4     if l < n and a[l] > a[m]: m = l
5     if r < n and a[r] > a[m]: m = r
6     if m == i: break
7     a[i], a[m] = a[m], a[i]; i = m
8 def heap_sort(a):
9     n = len(a)
10    for i in range(n//2-1, -1, -1): bajar(a, n, i) # O(n)
11    for i in range(n-1, 0, -1):
12        a[0], a[i] = a[i], a[0]; bajar(a, i, 0)
```

(En Java es idéntico al C++ con `a.length`.)

6.10 QuickSelect

Idea: encuentra el k -ésimo menor *sin ordenar todo*. Como quicksort, particiona alrededor de un pivote dejando a la izquierda lo menor y a la derecha lo mayor; pero como sabes en qué lado cae la posición k , recorres (o iteras) sobre *un solo* lado y descartas el otro. Por eso el promedio baja de $O(n \log n)$ a $O(n)$: cada fase procesa *la mitad* del anterior, y $n + \frac{n}{2} + \frac{n}{4} + \dots = 2n$. El peor caso $O(n^2)$ aparece con pivotes pésimos (arreglo casi ordenado + pivote extremo); **elegir el pivote al azar** lo vuelve prácticamente imposible. La versión de abajo es iterativa (sin pila de recursión). Si necesitas garantía $O(n)$ en el peor caso, existe *mediana de medianas* para elegir un buen pivote. **Complejidad:** promedio $O(n)$, peor $O(n^2)$.

```
1 int quickSelect(vector<int> a, int k){ // k in [0,n-1]
2 int lo=0, hi=a.size()-1;
3 while(lo<hi){
4     int p=a[lo+rand()%(hi-lo+1)], i=lo, j=hi; // pivote aleatorio
5     while(i<=j){ while(a[i]<p) i++; while(a[j]>p) j--;
6         if(i<=j){ swap(a[i],a[j]); i++; j--; } }
7     if(k<=j) hi=j; // k a la izq
8     else if(k>=i) lo=i; // k a la der
9     else return a[k];
10 }
11 return a[lo];
12 }
13 // C++ trae nth_element(a.begin(), a.begin()+k, a.end()); // O(n)
```

```
1 def quick_select(a, k): # k in [0, n-1]
2 lo, hi = 0, len(a)-1
3 while lo < hi:
4     p = a[random.randint(lo, hi)] # pivote aleatorio
5     i, j = lo, hi
6     while i <= j:
7         while a[i] < p: i += 1
8         while a[j] > p: j -= 1
9         if i <= j:
10            a[i], a[j] = a[j], a[i]; i += 1; j -= 1
11    if k <= j: hi = j
12    elif k >= i: lo = i
13    else: return a[k]
14 return a[lo]
```

(En Java igual, con `Random`; código completo en el repo.)

7 Búsqueda Binaria (Binary Search)

Idea central: mantienes un rango $[lo, hi]$ donde *sabes* que está la respuesta y en cada paso lo partes por la mitad, descartando la mitad que no puede contenerla. Como reduces a la mitad cada vez, llegas en $\log_2 n$ pasos. Funciona si los datos están **ordenados** o, más general, si existe un **predicado monótono** (una condición que pasa de falsa a verdadera una sola vez): entonces buscas la frontera del cambio. **Complejidad:** $O(\log n)$ en discreto, $O(\text{iter})$ en reales, $O(L \log n)$ con cadenas. Tip: usa $lo+(hi-lo)/2$ para el mid y evitar overflow.

7.1 En arreglo ordenado

Cómo: comparas x con el elemento del medio; si es menor, descartas la mitad derecha, si es mayor la izquierda. `lower_bound` te da el primer índice con valor $\geq x$ y `upper_bound` el primer $> x$; con eso sabes si x existe y cuántas copias hay.

```
1 int i = lower_bound(a.begin(), a.end(), x) - a.begin(); // >= x
2 int j = upper_bound(a.begin(), a.end(), x) - a.begin(); // > x
3 bool hay = binary_search(a.begin(), a.end(), x);
```

```
1 int idx = Arrays.binarySearch(a, x); // >=0 si existe
2 // lower_bound manual:
3 int lo=0, hi=a.length;
4 while (lo<hi){ int m=(lo+hi)>>1;
5   if(a[m]<x) lo=m+1; else hi=m; } // lo = primer >= x
```

```
1 from bisect import bisect_left, bisect_right
2 i = bisect_left(a, x) # primer >= x
3 j = bisect_right(a, x) # primer > x
```

7.2 Menor x que cumple

Cómo: aquí no buscas en un arreglo sino en el *rango de respuestas posibles*. El predicado es monótono F..F T..T (una vez que algo “sirve”, todo lo mayor también). En cada paso pruebas el `mid`: si cumple, lo guardas como candidato y sigues buscando a la izquierda (`hi=mid-1`) por si hay uno menor que también sirva; si no cumple, subes (`lo=mid+1`). Al final `res` es la frontera izquierda: el *menor* que cumple. Ejemplo: “mínima capacidad para repartir en $\leq k$ días”.

```
1 long long res=-1;
2 while(lo<=hi){ long long mid=lo+(hi-lo)/2;
3   if(cumple(mid)){res=mid; hi=mid-1;} // sirve -> izq
4   else lo=mid+1; }
```

```
1 long res=-1;
2 while(lo<=hi){ long mid=lo+(hi-lo)/2;
3   if(cumple(mid)){res=mid; hi=mid-1;}
4   else lo=mid+1; }
```

```
1 res = -1
2 while lo <= hi:
3     mid = (lo + hi) // 2
4     if cumple(mid): res = mid; hi = mid - 1
5     else: lo = mid + 1
```

7.3 Mayor x que cumple

Cómo: el caso espejo. Ahora el predicado es T..T F..F (sirve hasta cierto punto y luego deja de servir). Cuando el `mid` cumple, lo guardas y buscas a la *derecha* (`lo=mid+1`) por si hay uno mayor que también sirva; si no cumple, bajas (`hi=mid-1`). Respecto al anterior solo cambian esas dos líneas. Ejemplo: “máxima longitud de corte tal que salgan $\geq k$ piezas”.

```
1 long long res=-1;
2 while(lo<=hi){ long long mid=lo+(hi-lo)/2;
3   if(cumple(mid)){res=mid; lo=mid+1;} // sirve -> der
4   else hi=mid-1; }
```

```
1 long res=-1;
2 while(lo<=hi){ long mid=lo+(hi-lo)/2;
3   if(cumple(mid)){res=mid; lo=mid+1;}
4   else hi=mid-1; }
```

```
1 res = -1
2 while lo <= hi:
3     mid = (lo + hi) // 2
4     if cumple(mid): res = mid; lo = mid + 1
5     else: hi = mid - 1
```

7.4 Sobre reales

Cómo: cuando la respuesta es un número *real* (raíces, puntos de equilibrio) no hay “mitad entera” ni condición $lo \leq hi$ para parar. En su lugar iteras un número fijo de veces (unas 100 basta: el rango se divide por 2^{100} , precisión más que suficiente), moviendo `lo` o `hi` al `mid` según el predicado. Al terminar, $lo \approx hi$ es la frontera buscada.

```
1 for(int it=0; it<100; it++){ double mid=(lo+hi)/2;
2   if(cumple(mid)) hi=mid; else lo=mid; }
3 // lo - hi = frontera
```

```
1 for(int it=0; it<100; it++){ double mid=(lo+hi)/2;
2   if(cumple(mid)) hi=mid; else lo=mid; }
```

```
1 for _ in range(100):
2     mid = (lo + hi) / 2
3     if cumple(mid): hi = mid
4     else: lo = mid
```

7.5 Con strings

Cómo: exactamente la misma binaria, pero comparando cadenas en orden *lexicográfico* (diccionario) en lugar de números. La única diferencia de costo es que comparar dos cadenas cuesta $O(L)$ (longitud), no $O(1)$, así que el total sube a $O(L \log n)$. Ojo: el arreglo debe estar ordenado con el mismo criterio (ASCII: mayúsculas antes que minúsculas).

```
1 // vector<string> s ordenado
2 int i = lower_bound(s.begin(), s.end(), string("caro"))
3   - s.begin();
4 bool hay = binary_search(s.begin(), s.end(), string("caro"));
```

```
1 int idx = Arrays.binarySearch(s, "caro"); // usa compareTo
2 // manual: if(s[m].compareTo(x) < 0) lo=m+1; else hi=m;
```

```
1 i = bisect_left(s, "caro") # str comparan lexicografico
2 hay = i < len(s) and s[i] == "caro"
```

Parte IV Matemáticas

8 Matemáticas (Math)

Contenido en desarrollo.

9 Teoría de Números (Number Theory)

Contenido en desarrollo.

10 Combinatoria (Combinatorics)

Contenido en desarrollo.

11 Máscaras de Bits (Bitmasking)

Contenido en desarrollo.

12 Geometría (Geometry)

Contenido en desarrollo.

Parte V Técnicas Algorítmicas

13 Programación Dinámica (Dynamic Programming)

Contenido en desarrollo.

14 Algoritmos Voraces (Greedy)

Contenido en desarrollo.

15 Ad-hoc y Simulación

Contenido en desarrollo.

Parte VI Grafos y Árboles

16 Grafos (Graphs)

Contenido en desarrollo.

17 Árboles (Trees)

Contenido en desarrollo.

18 Disjoint Set Union (DSU)

Contenido en desarrollo.

Parte VII Cadenas

19 Cadenas (Strings)

Contenido en desarrollo.